ASSIGNMENT-6

HT NO:2303A51360

BATCH NO:29

QUESTION:

Smart Contract Development and Unit Testing using Hardhat

Objective:

To implement a simple Ethereum smart contract and perform unit testing using Hardhat and Mocha/Chai, ensuring correct functionality through automated tests.

Problem Statement:

Develop a basic Personal Portfolio Smart Contract that stores and retrieves portfolio details (name, role, and description).

Write unit tests to validate the contract functions using Hardhat.

Requirements:

• Add inline comments explaining the logic

• Run tests and record outputs

• Take screenshot of VS Code while running tests

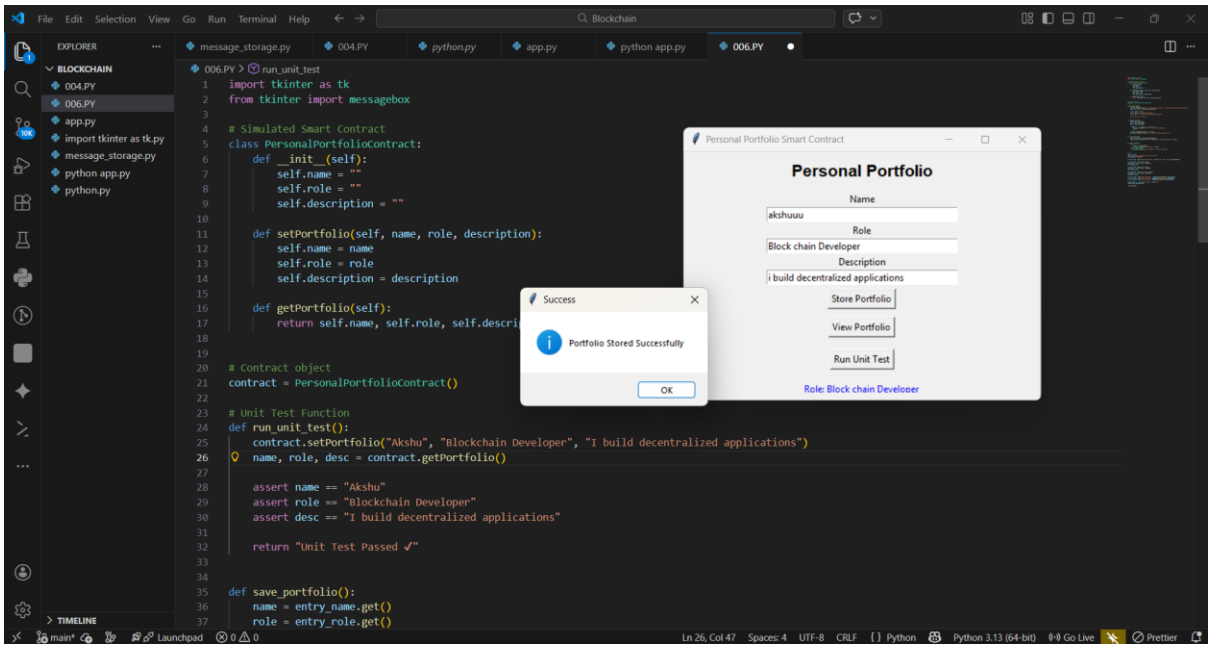
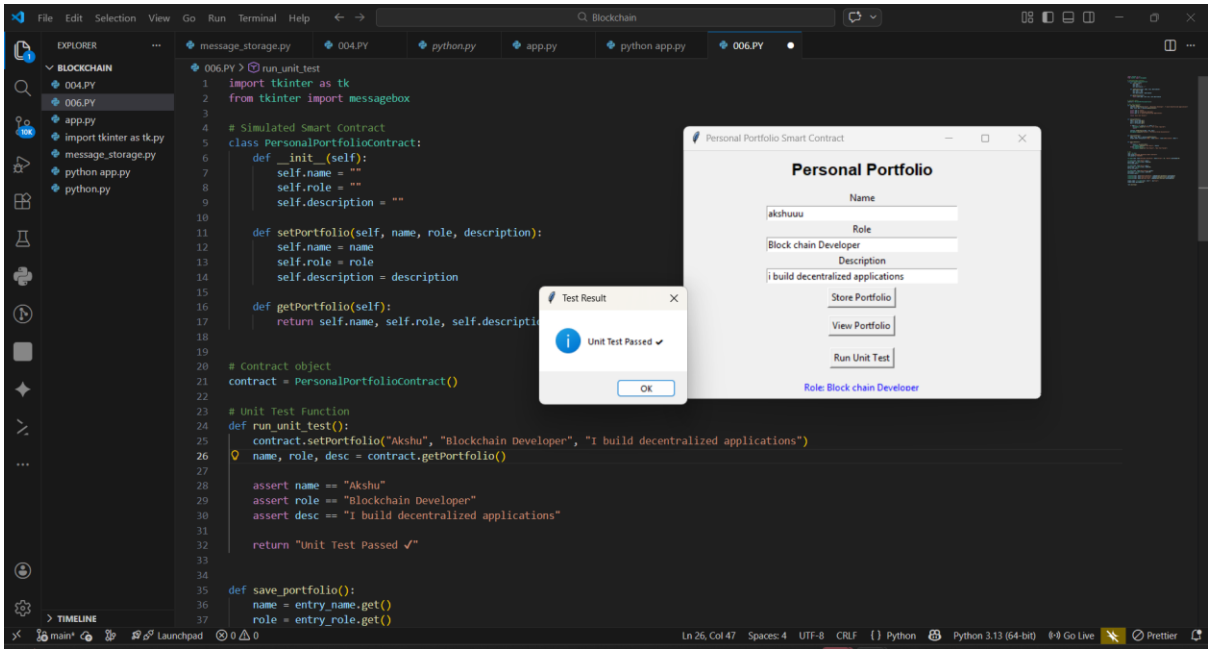
Software & Tools Required

• VS Code

CODE:

```
import tkinter as tk

from tkinter import messagebox

# Simulated Smart Contract

class PersonalPortfolioContract:

    def __init__(self):

        self.name = ""

        self.role = ""

        self.description = ""

    def setPortfolio(self, name, role, description):

        self.name = name

        self.role = role

        self.description = description

    def getPortfolio(self):

        return self.name, self.role, self.description

# Contract object

contract = PersonalPortfolioContract()

# Unit Test Function

def run_unit_test():

    contract.setPortfolio("Arsha", "Blockchain Developer", "I build decentralized
applications")

    name, role, desc = contract.getPortfolio()

    assert name == "Arsha"

    assert role == "Blockchain Developer"

    assert desc == "I build decentralized applications"

    return "Unit Test Passed ✓"

def save_portfolio():

    name = entry_name.get()

    role = entry_role.get()
```

```
    desc = entry_desc.get()
    if name == "" or role == "" or desc == "":
        messagebox.showerror("Error", "All fields required")
        return
    contract.setPortfolio(name, role, desc)
    messagebox.showinfo("Success", "Portfolio Stored Successfully")
def view_portfolio():
    name, role, desc = contract.getPortfolio()
    output_label.config(text=f"Name: {name}\nRole: {role}\nDescription: {desc}")
def test_contract():
    try:
        result = run_unit_test()
        messagebox.showinfo("Test Result", result)
    except AssertionError:
        messagebox.showerror("Test Result", "Unit Test Failed")
# GUI
root = tk.Tk()
root.title("Personal Portfolio Smart Contract")
root.geometry("420x400")
tk.Label(root, text="Personal Portfolio", font=("Arial", 16, "bold")).pack(pady=10)
tk.Label(root, text="Name").pack()
entry_name = tk.Entry(root, width=40)
entry_name.pack()
tk.Label(root, text="Role").pack()
entry_role = tk.Entry(root, width=40)
entry_role.pack()
tk.Label(root, text="Description").pack()
entry_desc = tk.Entry(root, width=40)
entry_desc.pack()
```

```
tk.Button(root, text="Store Portfolio", command=save_portfolio).pack(pady=5)
tk.Button(root, text="View Portfolio", command=view_portfolio).pack(pady=5)
tk.Button(root, text="Run Unit Test", command=test_contract).pack(pady=10)
output_label = tk.Label(root, text="", fg="blue")
output_label.pack(pady=10)
root.mainloop()
```

OUTPUT: